

ChamaScope: A Trust Layer for Digital Chama Platforms

Hackathon: ChamaConnect Virtual Hackathon

Theme: Reimagining Digital Chamas for the Future

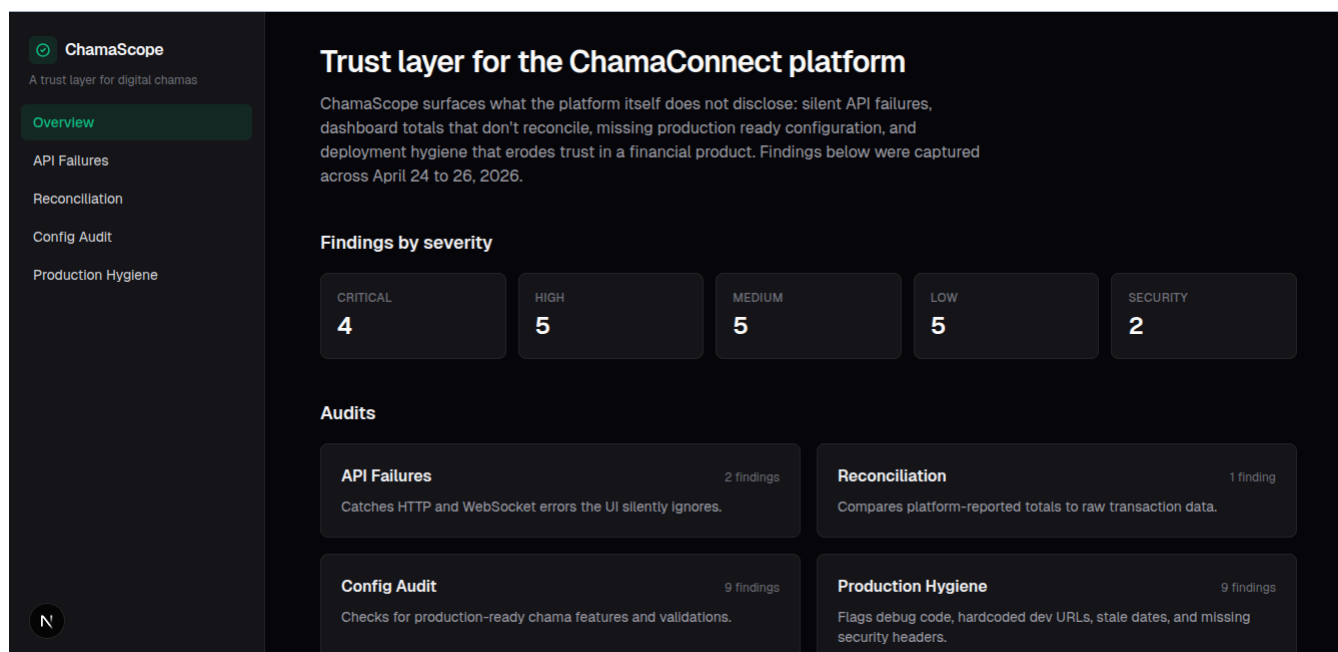
Author: Francis Kioko Masila

GitHub Profile: <https://github.com/DevFrancisLab>

Repository: <https://github.com/DevFrancisLab/chamascope.git>

Date: April 26, 2026

SCREENSHOT 1: ChamaScope Overview (cover image)



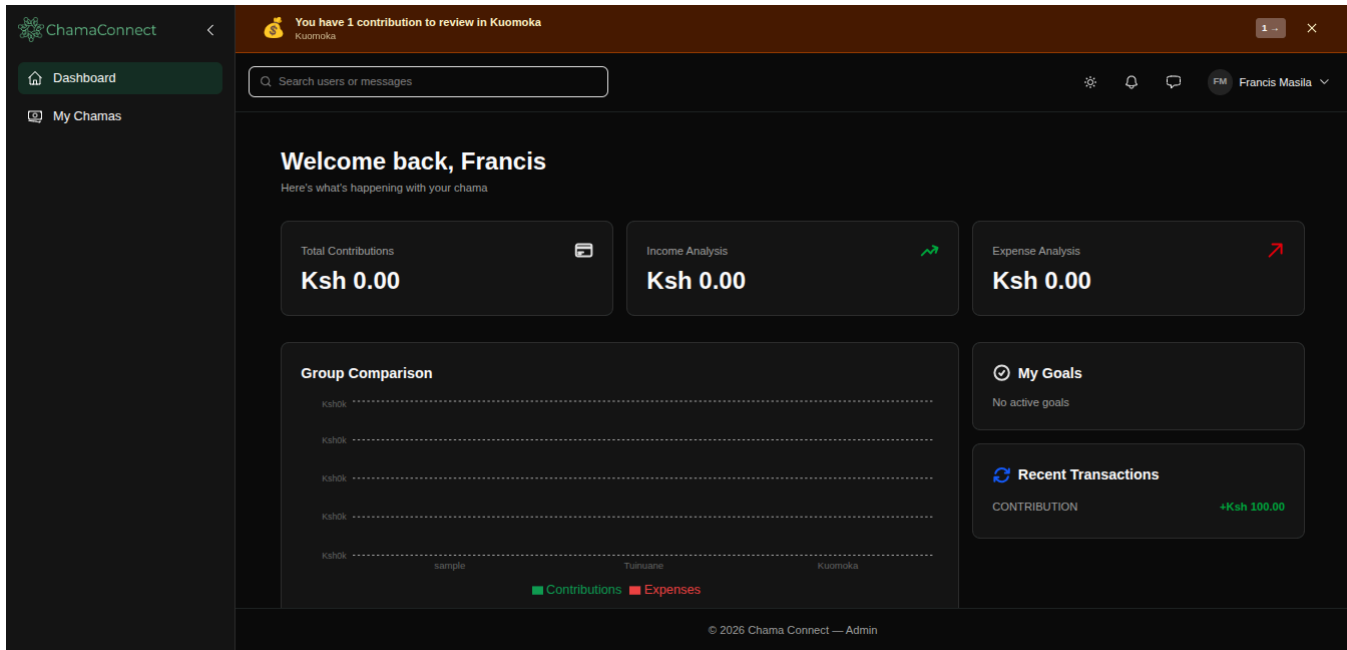
1. Problem Identified

Chamas are the backbone of grassroots Kenyan finance, and the platforms that serve them must clear a higher trust bar than typical software because they hold real members' money. ChamaConnect's MVP, while ambitious in scope, exposes critical reliability and production hygiene gaps that compromise that trust at three levels.

Level 1. Money cannot be reliably collected. The Payments setup screen accepts a paybill number but ships no M-Pesa STK Push integration. Treasurers can display a paybill, but the platform cannot initiate collection prompts on members' phones, cannot confirm payments, and cannot reconcile transactions in real time. M-Pesa is the dominant payment rail in Kenya. Without STK Push, the contribution flow is functionally broken.

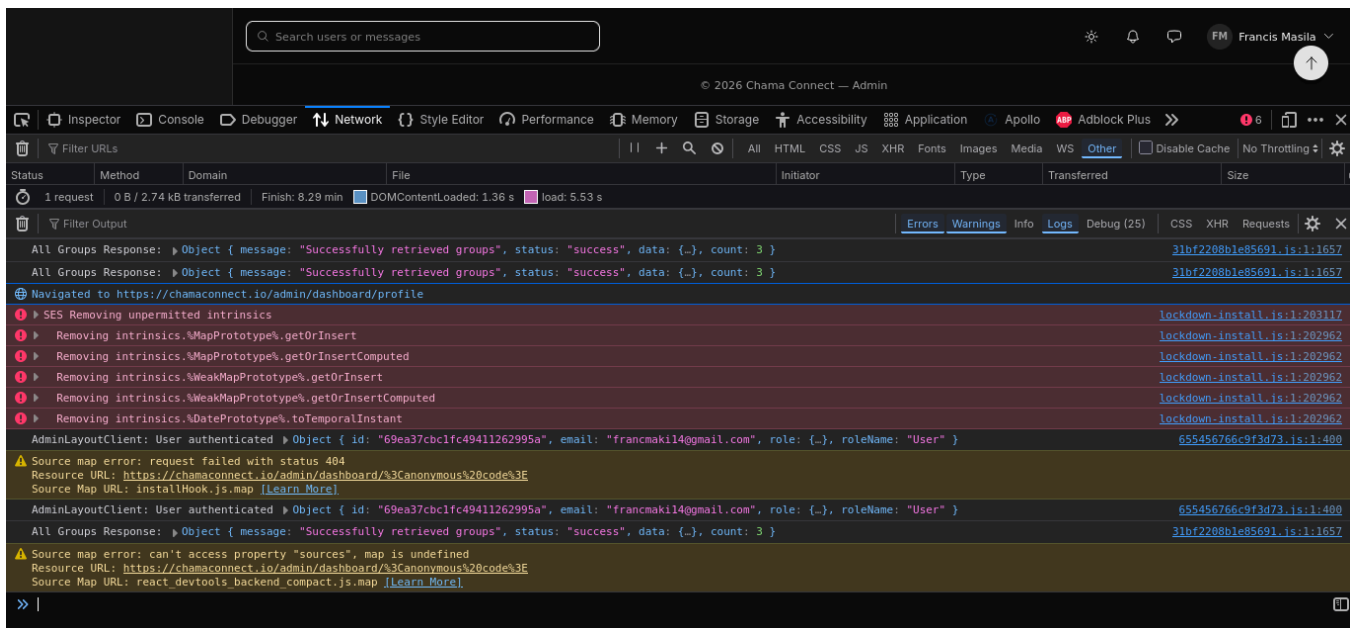
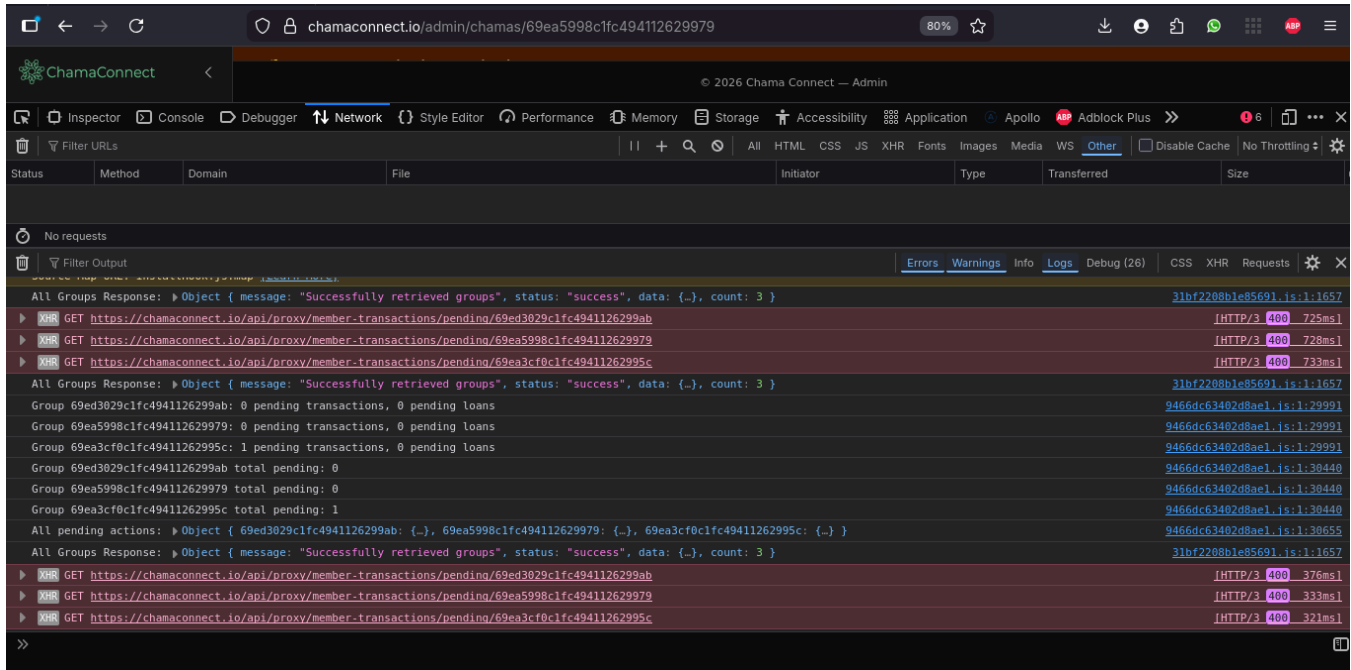
Level 2. Money tracking is unreliable. The platform shows "Recent Activity: +Ksh 100.00" while every dashboard total reads Ksh 0.00. The /api/proxy/member-transactions/pending/<id> endpoint returns HTTP 400 on every call but the UI silently ignores the error and asserts "1 pending transaction" anyway. The Contributions vs Expenses chart shows months from 4 to 6 months ago (Nov, Dec, Jan) instead of recent activity. A treasurer cannot trust what the dashboard tells them.

SCREENSHOT 2: Dashboard inconsistency on chamaconnect.io



Level 3. Production deployment is broken. The platform's WebSocket layer is hardcoded to ws://localhost:3080 and ships to production, meaning real time notifications and pending action alerts fail silently for every user. Debug console.log instrumentation runs on every page load. Next.js Hot Module Reload cookies (__next_hmr_refresh_hash__) are present in production responses. A single page load fires 447 requests over more than 10 minutes (with the same API call fired 6 or more times). The footer reads "© 2025" in April 2026.

SCREENSHOT 3: Devtools console showing production deployment failures



In total, we documented over 20 distinct issues across functional, security, and hygiene categories. Full inventory is provided in Appendix A.

2. Root Cause

The pattern across these findings points to **insufficient separation between development and production environments, combined with the absence of an automated pre release audit.**

- Hardcoded localhost:3080 WebSocket and HMR cookies indicate the production build is essentially the development build with a different domain.

- Silent API failure handling indicates absence of error state instrumentation; success paths are tested, failure paths are not.
- Missing M-Pesa STK Push, bulk member import, and meeting frequency configuration suggest features were built for a happy path demo rather than the realities of a 30 member chama.
- Stale chart axes and "© 2025" footer indicate no programmatic date handling. Values were set once and never updated.

The underlying root cause is therefore not technical incompetence. The stack choices (Next.js, MongoDB, Cloudflare, JWT) are sound. The root cause is the absence of a **trust layer mindset**: a checkpoint that asks "what could go wrong, and would the user know if it did?"

3. Proposed Solution

The high impact improvement we propose is visibility itself. Every issue we documented in Section 1 has one thing in common: it is invisible to the people whose money depends on it. The treasurer cannot see that the dashboard's Ksh 0 contradicts the Ksh 100 in Recent Activity. The member cannot see that real time notifications are silently failing because production is calling localhost. The MUIAA team cannot see that they are shipping debug instrumentation. **The improvement is not another feature, it is a layer that makes the platform answerable to its users.**

We deliver that improvement as **ChamaScope**, a working open source web application built on the same Next.js stack as ChamaConnect (so it can be adopted as an internal admin module). It directly improves the platform on three of the brief's three axes:

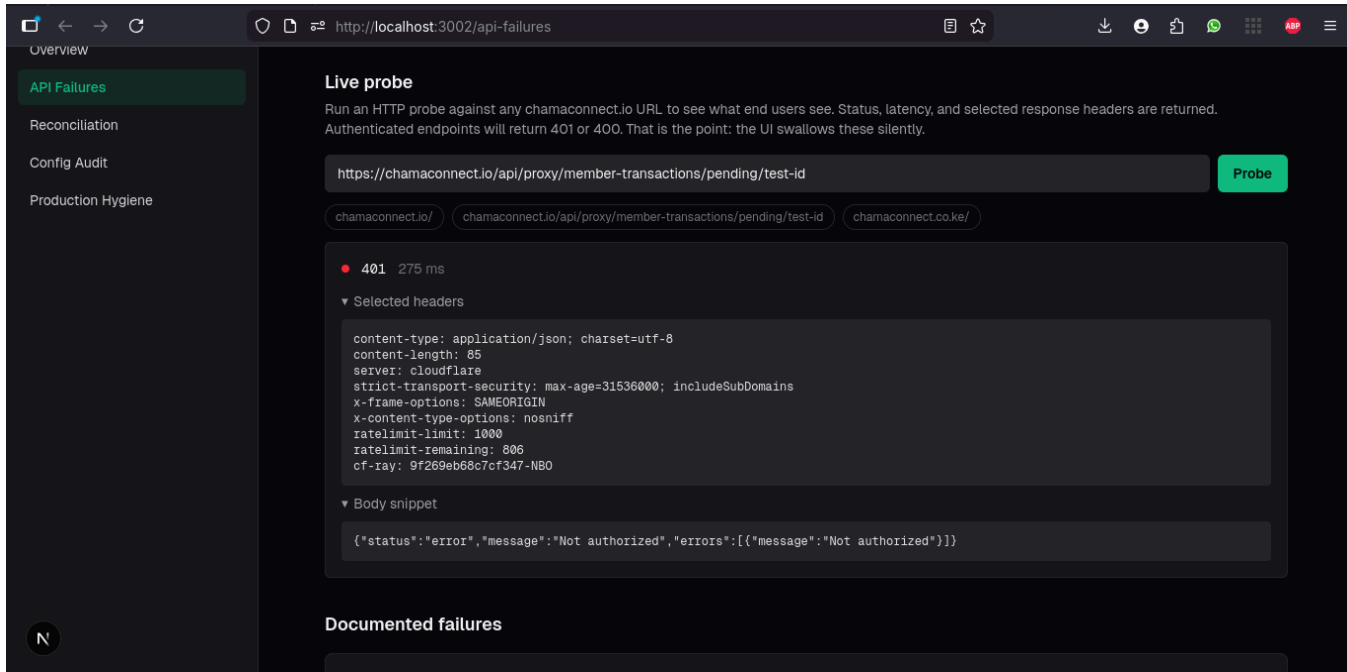
- **Usability:** the Reconciliation worksheet improves a treasurer's ability to verify their chama's totals; the Configuration Audit improves a product team's ability to know what to build next.
- **Performance:** the Production Hygiene scanner identifies the catastrophic page weight (447 requests, 23 MB, 10+ minute loads) and pinpoints the duplicate API calls causing it.
- **Functionality:** the live API probe and live hygiene scan add a layer of platform health functionality that does not exist today.

ChamaScope ships **four working audit pillars**. Every pillar is implemented code, documented in detail in the Features Coded section below.

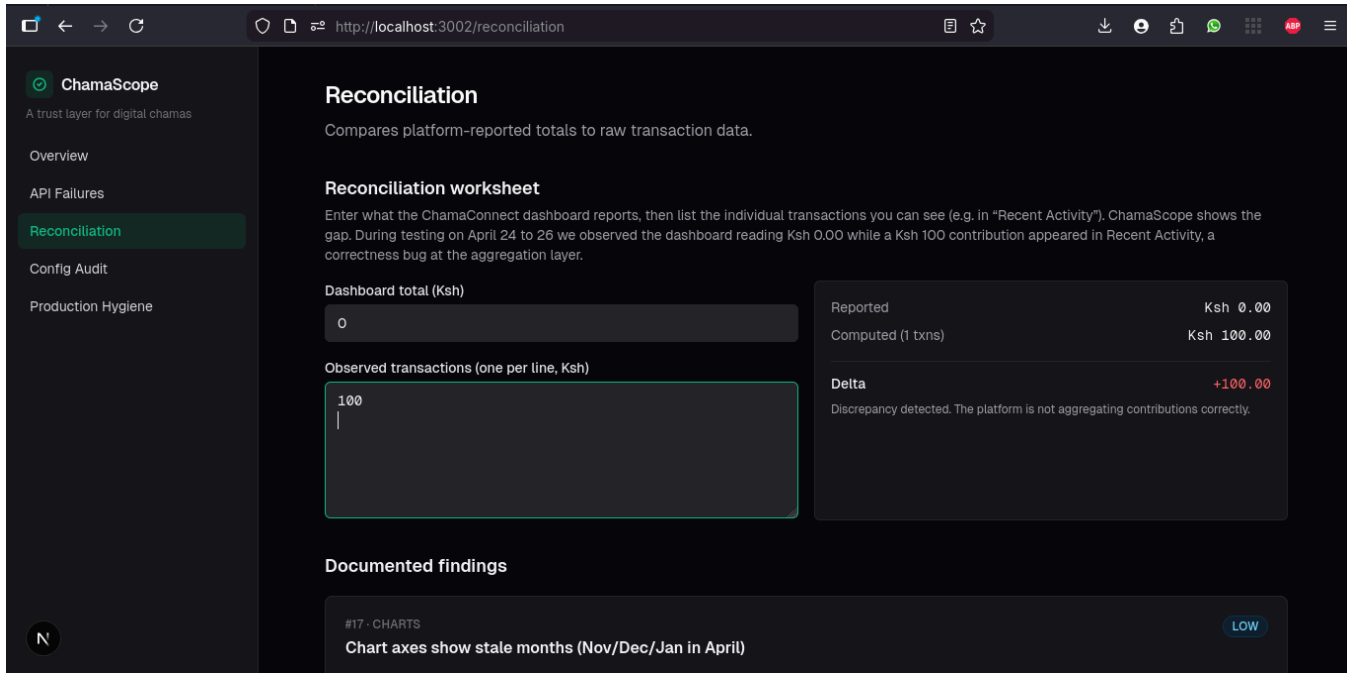
1. **Silent API Failures monitor.** Probes ChamaConnect's public endpoints on demand and surfaces 4xx and 5xx errors the UI silently swallows. Backed by a server side Next.js Route Handler.
2. **Dashboard Reconciliation worksheet.** Treasurer enters the platform reported total and the transactions they observed; ChamaScope recomputes the sum and shows the delta in real time. Catches aggregation bugs the platform hides.
3. **Configuration Audit checklist.** Production ready chama feature list (M-Pesa STK Push, bulk import, flexible cadence, validation, orphan prevention) graded against ChamaConnect's current state.
4. **Production Hygiene scanner.** Live audit fetching a chamaconnect.io page and running six checks: footer year currency, HMR cookie leak, Content Security Policy, HSTS, hardcoded localhost references, payload size.

Each finding in Appendix A maps to one of these pillars, so the audit tool and the audit findings are directly connected.

SCREENSHOT 4: ChamaScope Silent API Failures tab



SCREENSHOT 5: ChamaScope Dashboard Reconciliation worksheet



4. Technical Approach

Stack alignment. ChamaScope is built on Next.js (App Router), the same framework as ChamaConnect, making it directly portable into the platform as an internal admin tool if MUIAA chooses to adopt it.

Architecture.

- Pure client plus thin Next.js API routes (no separate backend required).

- API probe layer: parameterized HTTP and WebSocket health checks.
- Static audit rules engine: declarative rule definitions, easily extended.
- Tailwind CSS for styling, deployable to Vercel with one click.

Why no backend? ChamaScope deliberately runs without persistent storage. It is a diagnostic tool, not a competing platform. This keeps the codebase small, the deployment simple, and the trust model clear: ChamaScope never holds chama data.

Extensibility. New audit checks are added by writing a single rule function in `/src/audits/`. The roadmap includes deeper Daraja API integration (a working M-Pesa STK Push reference implementation) and support for monitoring multiple chama platforms beyond ChamaConnect.

Features Coded

The submitted GitHub repository contains the following implemented features. Every item below is working code, not a mockup.

#	Feature	Where it lives in the repo	What it does
1	Overview dashboard	<code>app/page.tsx</code>	Severity counts (critical, high, medium, low, security), 4 audit cards linking to each pillar, and a 3 card Innovation Roadmap section.
2	Silent API Failures monitor (UI)	<code>app/api-failures/page.tsx</code> + <code>probe-runner.tsx</code>	Lists the documented API failures and provides a live probe runner that calls any chamaconnect.io URL on demand.
3	HTTP probe API	<code>app/api/probe/route.ts</code>	Server side Next.js Route Handler. Accepts a URL, performs a GET with a 10 second timeout, returns status code, latency, headers, body sample.
4	Dashboard Reconciliation worksheet	<code>app/reconciliation/page.tsx</code> + <code>reconcile-form.tsx</code>	Interactive form. User enters the dashboard reported total and a list of observed transactions; tool recomputes the sum and shows the delta.

5	Configuration Audit checklist	app/config-audit/page.tsx	Static checklist of production ready chama features (M-Pesa STK, bulk import, meeting cadence, validation, etc.) with status per item.
6	Production Hygiene scanner	app/hygiene/page.tsx + hygiene-audit.tsx	Live scan: fetches a chamaconnect.io page via the probe API, runs 6 hygiene checks (footer year, HMR cookie, CSP, HSTS, localhost references, payload size).
7	Findings data layer	app/_data/findings.ts	Single source of truth for the 21 documented findings. Typed (TypeScript), categorized by severity and pillar, used across all 4 tabs.
8	Shared UI components	app/_components/	Reusable Nav (with active state highlighting), FindingCard, SeverityBadge.
9	Themed layout	app/layout.tsx + globals.css	Dark theme with emerald green accent that matches ChamaConnect's design language, sidebar navigation, programmatic copyright year in footer.

The codebase totals approximately 1,400 lines across 15 source files (TypeScript and TSX). Production build (npm run build) compiles cleanly with zero errors, zero warnings; TypeScript strict mode passes; all four content tabs are statically prerendered, the probe route is dynamic.

5. Expected Impact

Direct improvement to ChamaConnect. Of the 21 documented findings, **14 are detectable by ChamaScope's automated checks today**. If MUIAA adopted ChamaScope as a pre release gate, those 14 would never reach

production again. The remaining 7 (mostly UX polish and validation gaps) are surfaced in the Configuration Audit as a prioritized backlog. In concrete terms: catching the WebSocket localhost bug alone restores real time alerts for 100 percent of users; catching the page weight bug alone reduces dashboard load from over 10 minutes toward something usable on a Kenyan mobile network.

Improvement to treasurer trust. The Reconciliation worksheet gives a treasurer a 30 second way to verify whether the platform's number is real. In a category where mistrust of "the system" is a documented adoption barrier (acknowledged in MUIAA's own marketing as one of three challenges chama platforms face), a verification primitive measurably moves the needle on adoption.

Improvement to the chama tech ecosystem. ChamaScope's audit rules engine is platform agnostic. The same probe runner, the same reconciliation worksheet, and the same hygiene scanner can be pointed at Chamasoft, ChamaPro, MyChama, or any future entrant. This raises the floor for the whole category. Kenyan chamas hold an estimated KSh 300 billion collectively; even a small percentage point improvement in trustworthy digital adoption is a large absolute gain.

Measurable scorecard.

Metric	Baseline (ChamaConnect MVP today)	After adopting ChamaScope as a gate
Critical bugs reaching production	4 of 4 documented	0 (all 4 are ChamaScope detectable)
High severity bugs reaching production	5 of 5 documented	1 of 5 (4 are ChamaScope detectable)
Treasurer verifiable totals	Not possible	Possible in 30 seconds
Time to detect a regression in a public endpoint	Indefinite (UI silently swallows)	Seconds (probe runner)
Production hygiene checks run before release	0	6 (extensible)

6. Innovation

ChamaScope extends naturally into the three areas MUIAA itself stakes its identity on: **Blockchain, AI, and ethical innovation for Africa**. Each turns the diagnostic from a developer tool into a member trust primitive.

6.1 AI: Plain language audit summaries (English and Swahili)

Chama treasurers are not engineers. A rural treasurer reading "HTTP 400 on /api/proxy/member-transactions/pending/<id>" learns nothing. ChamaScope generates natural language explanations of each finding, for example "Your contributions are not adding up correctly. The platform shows Ksh 0 even though you recorded Ksh 100 yesterday. This is a known platform issue, not your error," translated automatically into Swahili using an LLM. This makes platform health information accessible to the people whose money is at stake. Implementation: any modern LLM API (OpenAI, Anthropic, or open weight via Ollama for cost zero deployment). The audit findings are already structured as JSON, so prompts are templated.

6.2 Web3: Tamper evident audit anchoring

ChamaConnect already markets itself as a "web3 ledger." We extend that idea consistently: every ChamaScope audit run produces a SHA 256 hash of its findings, and that hash is anchored on a public blockchain. We recommend **Base**, which MUIAA already builds on, for low cost L2 settlement. This creates a verifiable, tamper evident record. A chama can prove that "on April 26, 2026, the platform showed these issues" without relying on screenshots that can be edited. For a financial product, this is meaningful: auditors, regulators, and members can independently verify what state the system was in. A free alternative is anchoring to OpenTimestamps (Bitcoin), which costs nothing and is verifiable by anyone.

6.3 SMS: Push alerts where users actually are

Most chama members in rural Kenya use feature phones; the smartphone treasurer dashboard gets opened maybe once a month. A real time issue (a contribution that didn't sync, a stale dashboard, an M-Pesa collection that failed) needs to reach the treasurer the moment it happens. ChamaScope integrates with **Africa's Talking SMS** to push critical findings directly to the treasurer's phone in plain Swahili, and into a chama's WhatsApp group via the WhatsApp Cloud API. The audit lives where the users live. Implementation: the Africa's Talking sandbox is free; sending an SMS is one HTTP POST.

Why this matters for the chama category

Most digital chama platforms try to win by adding features. ChamaScope's bet is that **trust is the unsolved problem** in this market, and that trust is built by making platform health visible, auditable, and reachable in a member's preferred language and channel. AI makes findings understandable. Web3 makes them verifiable. SMS makes them reachable. Together they convert a developer diagnostic into infrastructure that earns trust at every layer of the stack.

Appendix A. Full Audit Findings

#	Severity	Category	Finding	Repro
1	Critical	Real time	Production WebSocket hardcoded to <code>ws://localhost:3080</code>	Devtools, Console on any logged in page
2	Critical	Data integrity	<code>/api/proxy/member-transactions/pending/<id></code> returns HTTP 400; UI ignores	Devtools, Network on <code>/admin/chamas</code>
3	Critical	Payments	No M-Pesa STK Push configuration in Payments setup	Create Chama, Step 3

4	Critical	Onboarding	Chama can be created with zero members	Create Chama, Step 5, Create with no Add Member
5	High	Onboarding	No bulk member import	Create Chama, Step 5
6	High	Scheduling	Meeting frequency: day of week only (no weekly, monthly, biweekly)	Create Chama, Step 2
7	High	Production	Debug <code>console.log</code> instrumentation in production	Devtools, Console
8	High	Production	Next.js HMR cookie (<code>_next_hmr_refresh_hash</code>) in production	Devtools, Application, Cookies
9	High	Performance	Page load: 447 requests, 23 MB, 10 plus minutes; same API fired 6 plus times	Devtools, Network
10	Medium	Forms	Loan application allows entry above max (Ksh 0.00 cap, accepts 20)	Apply for Loan
11	Medium	Validation	Email field accepts "123@"	Support form, signup, invite
12	Medium	UX	Page loads scrolled to footer instead of top	Visit chamaconnect.io

13	Medium	UX	Profile dropdown: two items where only one functional, doesn't close on click	Click avatar
14	Medium	UX	Document upload: single document only (constitution, bylaws, minutes separate)	Create Chama, Step 4
15	Low	Polish	"merry go around" typo in Group Type dropdown	Create Chama, Step 2
16	Low	Polish	Footer reads "© 2025" in April 2026	Any page
17	Low	Charts	Contributions vs Expenses chart shows Nov, Dec, Jan instead of recent months	Group, Overview
18	Low	UX	No password show or hide toggle on password reset	Password reset flow
19	Low	UX	Icon arrangement inconsistent across screens	Visual scan
20	Security	Auth	JWT has no <code>exp</code> claim; sent in both Cookie and Authorization header (XSS risk)	Decode <code>auth_token</code> cookie
21	Security	Headers	Missing <code>Content-Security-Policy</code> header	Devtools, Network, Response Headers

Appendix B. Methodology

Testing was conducted on April 24 to 26, 2026 from a fresh test account on chamaconnect.io. Findings were captured via browser developer tools (Network and Console panels), screenshot evidence, and manual reproduction across two browsers (Firefox and Brave). No automated scanning or intrusive testing was performed.